

Personal Wealth OS — 代码质量与体验分析报告

分析日期: 2026-08-17 | 版本: v0.2.0 | 技术栈: Vite 5 · TypeScript · Firebase ·
Lightweight Charts

目录

- 项目总览与架构评估
- 模块职责与文件分析
- 代码质量分析
- 安全性分析
- 用户体验 (UX) 分析
- 可改进模块与建议
- 总结评分与下一步

1. 项目总览与架构评估

12

核心源文件

2,752

ui.ts 行数

11

功能页面

14

状态版本号

架构概述: 项目采用原生 TypeScript + DOM 操作的单页应用架构, 模块按职责清晰划分:

`models.ts` (领域模型)、`state.ts` (状态管理与持久化)、`rules.ts` (纯函数计算)、`market.ts` (行情数据)、`firebase.ts` (认证与云同步)、`ui.ts` (页面渲染与事件绑定)、`main.ts` (启动与顶层协调)。

✓ **良好** 模块职责边界清晰, `rules.ts` 保持纯函数设计, 不直接操作 DOM 或存储。

✓ **良好** 状态迁移机制完善, `migrateState()` 函数覆盖了版本 3 到 14 的所有迁移路径。

✓ **良好** PWA 支持完整, 含 Service Worker、manifest、离线提示和 iOS 安装引导。

2. 模块职责与文件分析

文件	行数	职责	状态
<code>models.ts</code>	~200	领域模型、接口定义 (WealthState, Trade, Goal, Bucket 等)	良好
<code>state.ts</code>	633	默认状态、持久化、版本迁移、快照系统、导入导出	良好

rules.ts	223	财务计算（盈亏、现金流、紧急基金、投资组合摘要）	良好
market.ts	672	行情获取、基本面、历史价格、风险指标、交易时间线	可优化
firebase.ts	110	认证（Google OAuth）、Firestore 读写与订阅	需注意
ui.ts	2,752	全部页面模板、事件绑定、导航、图表渲染	需拆分
main.ts	274	启动流程、认证状态管理、主题、路由	良好
financialHealth.ts	~300	净资产、月结、经常性交易预测、目标关联	良好
ledger.ts	~400	账本分类汇总、账户余额、过滤器	良好
styles.css	大	全局样式、响应式、深色/浅色主题、组件样式	可优化

3. 代码质量分析

3.1 优点

- **严格 TypeScript:** `tsconfig.json` 启用 `strict: true` , 目标 ES2022, 代码中几乎无 `any` 使用 (仅 `unknown` + 类型收窄) 。
- **纯函数设计:** `rules.ts` 中所有计算函数均为纯函数, 接受 `WealthState` 并返回结果, 无副作用, 易于测试。
- **状态不可变:** `setState()` 使用不可变语义, 通过结构化克隆避免引用污染。
- **数据安全:** 账本交易验证函数 `validLedgerTransactions()` 对每条记录进行严格校验 (金额、日期、类型、账户引用) 。
- **防御性编程:** 市场数据模块对 Yahoo Finance API 返回结构做多层空值检查, 缓存失效机制清晰。

3.2 发现的问题

问题 #1: `ui.ts` 过大 (2,752 行)

单文件包含 11 个功能页面的全部模板生成和事件绑定逻辑, 违反单一职责原则。新增或修改任何页面时需要在巨大文件中定位, 维护成本高且容易引入回归。

问题 #2: 硬编码汇率

`market.ts` 中 `calcPnLForTicker()` 默认汇率为 `usdToMyr = 4.25` , `buildTradeTimelineHtml()` 也硬编码 4.25。汇率应从用户配置或动态 API 获取。

问题 #3: 缺少单元测试

项目完全没有自动化测试。`rules.ts` 中的成本基础计算 (FIFO 卖出)、`state.ts` 中的迁移逻辑和 `market.ts` 中的风险指标计算均无测试覆盖。对于金融计算类应用, 这是一个显著的风险。

问题 #4: CORS 代理依赖脆弱

`market.ts` 依赖 3 个公共 CORS 代理 (`allorigins`、`corsproxy`、`codetabs`) , 这些第三方服务随时可能不可用或限流。建议部署自建代理或改用后端 API 路由。

问题 #5: HTML 模板字符串中的 XSS 风险

虽然大部分用户输入经过 `escapeHtml()` 处理, 但 `ui.ts` 中有大量 `innerHTML` 赋值, 且部分模板函数直接拼接来自 API 的数据 (如 `market.ts` 的 `buildTradeTimelineHtml()` 中的 `shortName`、`ticker`) , 存在潜在 XSS 风险。

问题 #6: `saveState` 静默忽略无 UID 的保存

`state.ts` 中 `saveState()` 在 `!uid` 时直接 `return` , 不做任何提示。如果调用方忘记传入 UID, 用户的数据将不会被持久化, 且无任何错误反馈。

问题 #7: Beta 计算前提假设不安全

`calcRiskMetrics()` 中计算 Beta 时假设 `benchmarkPrices.length` $\equiv$

`prices.length`，如果不相等则默认 `beta = 1`。实际场景中两个时间序列长度经常不一致，应做日期对齐。

4. 安全性分析

项目	状态	说明
Firebase API Key	正常	客户端配置中的 API Key 是 Firebase Web SDK 的公开配置，安全性由 Firestore Rules 保障
Firestore Rules	已配置	存在 <code>firestore.rules</code> 文件，用户只能访问 <code>/users/{uid}/wealth/state</code>
XSS 防护	部分	有 <code>escapeHtml()</code> 但覆盖面不完全，API 数据直接拼接风险存在
敏感数据	良好	日志中不记录认证令牌；隐私遮罩功能支持金额隐藏
输入校验	良好	状态迁移和导入文件均有严格的校验和回退逻辑

5. 用户体验 (UX) 分析

5.1 优点

- **信息架构清晰**: 页面按 Today / Plan / Grow / Reflect / System 五个分组组织, 符合用户心智模型。
- **深色/浅色主题**: 支持一键切换, 使用 CSS 变量实现, 对比度合理。
- **PWA 安装体验**: Android 和 iOS 均有安装引导, iOS 提供分步图文指引。
- **快照系统**: 每次重大操作前自动保存快照 (最多 20 个), 用户可随时回滚, 降低了操作风险。
- **Quick View**: 提供 `/quick` 路径的快速查看模式, 适合 PWA 快速启动。
- **隐私遮罩**: `maskAmounts` 功能一键隐藏所有金额, 适合公共场合使用。

5.2 需改进的体验问题

UX 问题 #1: 市场数据加载无骨架屏

行情数据通过 CORS 代理获取, 可能需要 5-15 秒。在等待期间用户看不到任何加载指示器 (取决于 UI 实现), 容易误以为应用无响应。

UX 问题 #2: 错误状态不一致

市场数据获取失败时 (网络超时、代理不可用、API 返回异常), 用户可能只看到空白区域而无明确的错误提示和重试按钮。

UX 问题 #3: 移动端表单体验

财务输入表单使用 , 在 iOS 上会弹出数字键盘但失去小数点便捷输入; 建议使用 `inputmode="decimal"` 。

UX 问题 #4: 首次用户引导缺失

新用户登录后直接看到默认状态数据 (MYR 880 allowance 等), 没有引导流程帮助理解各模块功能或进行初始配置。

6. 可改进模块与建议

[高优先级] #1 — 拆分 ui.ts

建议: 将 `ui.ts` 按页面拆分为独立模块:

`ui/dashboard.ts`、`ui/portfolio.ts`、`ui/ledger.ts`、`ui/market.ts`、
`ui/settings.ts`、`ui/goals.ts`、`ui/buckets.ts`、`ui/advisor.ts`、
`ui/rules.ts`、`ui/review.ts`、`ui/calculator.ts`

保留 `ui.ts` 作为入口, 负责导航、侧边栏和 `renderApp()` 调度。每个子模块导出自己的 `render()` 和 `bind()` 函数。

[高优先级] #2 — 添加金融计算单元测试

建议：使用 Vitest 为以下核心函数编写测试：

- `calculatePositionCostBasis()` — FIFO 卖出、部分卖出、零持仓边界
- `migrateState()` — 版本 3→14 的所有迁移路径
- `calcRiskMetrics()` — 空序列、单点、正常数据、Beta 对齐
- `portfolioSummary()` — 空持仓、单 ticker、多 ticker 配置

[高优先级] #3 — 动态汇率替换硬编码

建议：将 USD/MYR 汇率存入 `WealthState.profile`，默认从用户交易的 `exchangeRate` 字段取最新值，或通过 API 获取实时汇率。避免所有 4.25 硬编码。

[中优先级] #4 — 自建 CORS 代理或后端 API

建议：项目已有 Vercel API 路由（`api/` 目录），可将行情请求移到后端 serverless function，消除对公共 CORS 代理的依赖。这也便于添加 API 缓存层（如 Vercel KV 或 Redis）。

[中优先级] #5 — 增强加载和错误状态 UI

建议：

- 为所有异步数据加载区域添加骨架屏（skeleton loader）
- 网络请求失败时显示可操作的错误消息 + 重试按钮
- 缓存过期数据仍可展示，但需标注“数据时间”和“离线/陈旧”状态

[中优先级] #6 — XSS 防护加固

建议：

- 审计 `ui.ts` 和 `market.ts` 中所有 `innerHTML` 赋值点
- 对来自 API 的数据（`shortName`、`symbol`、`marketState`）统一经过 `escapeHtml()`
- 考虑使用 `textContent` 替代 `innerHTML` 作为默认策略

[低优先级] #7 — 首次用户引导流程

建议：新增用户首次登录后显示简短的欢迎引导（3-5 步），帮助理解：Dashboard 概览 → Money Plan 分桶 → Investments 记录首笔交易 → Coach 查看建议。

[低优先级] #8 — Beta 日期对齐优化

建议：在 `calcRiskMetrics()` 中按日期对齐两个时间序列（取交集日期），避免因交易日不同导致长度不一致时 Beta 默认为 1 的问题。

7. 总结评分

7.5 / 10 综合代码质量评分

维度	评分	说明
架构设计	8.5 / 10	模块划分清晰，职责边界良好；ui.ts 是主要扣分项
代码质量	8.0 / 10	TypeScript 严格模式、纯函数设计；缺少测试是最大短板
数据安全	8.0 / 10	状态迁移健壮、校验充分；saveState 静默失败需改进
安全性	7.0 / 10	认证流程完整；HTML 拼接中的 XSS 风险需要加固
用户体验	7.5 / 10	导航结构好、PWA 支持完整；加载状态和错误反馈可优化
可维护性	6.5 / 10	ui.ts 2752 行是维护瓶颈；无测试降低了重构信心

下一步行动清单

- 高** — 拆分 `ui.ts` 为按页面组织的子模块
- 高** — 引入 Vitest，为 `rules.ts` 和 `state.ts` 的迁移逻辑编写测试
- 高** — 替换硬编码汇率 4.25 为可配置/动态值
- 中** — 将行情请求迁移到 Vercel serverless function
- 中** — 审计并加固所有 `innerHTML` 中的 XSS 防护
- 中** — 为异步数据区域添加骨架屏和错误重试 UI
- 低** — 设计首次用户引导流程
- 低** — 优化 Beta 计算的日期对齐逻辑